

On forests and trees

Przemysław Gordinowicz

Institute of Mathematics,
Łódź University of Technology



Graphs & Social Networks
Lecture 3

Outline



- 1 Motivation
- 2 Cut-vertices and cut-edges
- 3 Definition and basic properties
- 4 Spanning trees and counting trees
- 5 Applications of trees

Outline



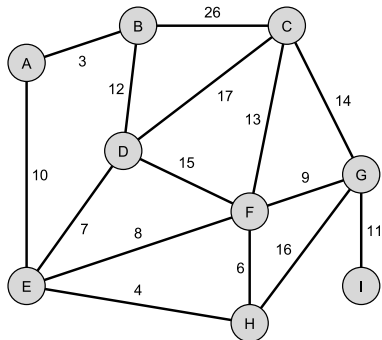
- 1 Motivation
- 2 Cut-vertices and cut-edges
- 3 Definition and basic properties
- 4 Spanning trees and counting trees
- 5 Applications of trees

The problem of railway builders



Problem (cf. D. Harel „Algorithmics. The spirit of computing ”)

*Knowing the cost of building a direct rail connection between each pair of cities (for which such a connection is possible), design the **cheapest** rail network connecting **all** cities.*

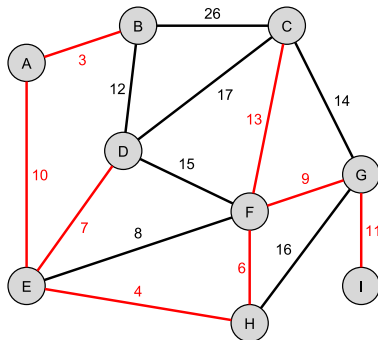


The problem of railway builders



Problem (cf. D. Harel „Algorithmics. The spirit of computing ”)

*Knowing the cost of building a direct rail connection between each pair of cities (for which such a connection is possible), design the **cheapest** rail network connecting **all** cities.*



Outline



- 1 Motivation
- 2 Cut-vertices and cut-edges**
- 3 Definition and basic properties
- 4 Spanning trees and counting trees
- 5 Applications of trees

Reminder



Definition

Vertices $u, v \in V$ of undirected graph G are **connected** if there exists uv -path in G , that means a path $v_0, \dots, v_k$ such that $v_0 = u, v_k = v$. A graph with pairwise connected vertices is called **connected graph**.

Definition

Each maximal (thus induced) connected subgraph of graph G is called its **connected component**.

Cut-vertices and cut-edges



Definition

Let G be an undirected graph. We say that vertex $v \in V(G)$ is a **cut-vertex** in G , when graph $G \setminus v$ has more connected components than graph G .

Analogously we say that edge $e \in E(G)$ is a **cut-edge** (or a **bridge**) of graph G , when graph $G - e$ has more more connected components than graph G .

Picture :-)



Cut-vertices and cut-edges

Definition

Let G be an undirected graph. We say that vertex $v \in V(G)$ is a **cut-vertex** in G , when graph $G \setminus v$ has more connected components than graph G .

Analogously we say that edge $e \in E(G)$ is a **cut-edge** (or a **bridge**) of graph G , when graph $G - e$ has more more connected components than graph G .

Theorem

An edge $e \in E(G)$ of graph G is a cut-edge if and only if e is not a part of any cycle in G .

Outline



- 1 Motivation
- 2 Cut-vertices and cut-edges
- 3 Definition and basic properties**
- 4 Spanning trees and counting trees
- 5 Applications of trees

Definitions



Definition

Graph that contains no cycle is called to be **acyclic** or a **forest**.
Connected acyclic graph is called a **tree**).

Definition

A vertex of degree 1 is called a **leaf**.

Picture :-)



Definitions

Definition

Graph that contains no cycle is called to be **acyclic** or a **forest**.
Connected acyclic graph is called a **tree**).

Definition

A vertex of degree 1 is called a **leaf**.

Picture :-)

Definitions



Definition

Graph that contains no cycle is called to be **acyclic** or a **forest**.
Connected acyclic graph is called a **tree**).

Definition

A vertex of degree 1 is called a **leaf**.

Picture :-)

Counting edges

Lemma

Every (finite) tree on at least 2 vertices has at least 2 leaves. Deleting a leaf from an n -vertex tree produces a tree on $(n - 1)$ vertices.

Theorem

For any multigraph G on n vertices ($n \geq 1$) the following are equivalent:

- ① *G is a tree;*
- ② *G is connected and has $(n - 1)$ edges;*
- ③ *G has $(n - 1)$ edges and no cycles;*
- ④ *G has no loops and has, for each $u, v \in V(G)$, exactly one u, v -path.*

Counting edges

Lemma

Every (finite) tree on at least 2 vertices has at least 2 leaves. Deleting a leaf from an n -vertex tree produces a tree on $(n - 1)$ vertices.

Theorem

For any multigraph G on n vertices ($n \geq 1$) the following are equivalent:

- ① *G is a tree;*
- ② *G is connected and has $(n - 1)$ edges;*
- ③ *G has $(n - 1)$ edges and no cycles;*
- ④ *G has no loops and has, for each $u, v \in V(G)$, exactly one u, v -path.*

Counting edges



Corollary

- *Every edge of a tree is a cut-edge.*
- *Adding one edge to a tree forms exactly one cycle.*

Outline



- 1 Motivation
- 2 Cut-vertices and cut-edges
- 3 Definition and basic properties
- 4 Spanning trees and counting trees**
- 5 Applications of trees

Question



It is obvious, that there exists $2^{\binom{n}{2}}$ simple graphs on vertex set $[n]$, where $[n] = \{1, 2, \dots, n\}$.

How much of them are trees?

Or (in other words) how many trees on vertex set $[n]$ exists?

Question



It is obvious, that there exists $2^{\binom{n}{2}}$ simple graphs on vertex set $[n]$, where $[n] = \{1, 2, \dots, n\}$.

How much of them are trees?

Or (in other words) how many trees on vertex set $[n]$ exists?

Spanning trees



Definition

Let G be a graph, and G_0 its subgraph. We say that G_0 is **spanning subgraph** of G , when $V(G) = V(G_0)$.

Definition

A **spanning tree** is a spanning subgraph that is a tree.

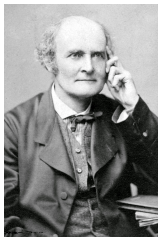
Observation (of „railway builders”)

Every connected graph has a spanning tree.

Counting trees vs spanning trees



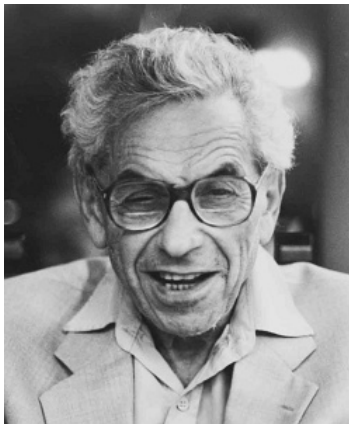
To count all trees on vertex set $[n]$ one can (equivalently) count all spanning trees of the complete graph on set $[n]$.



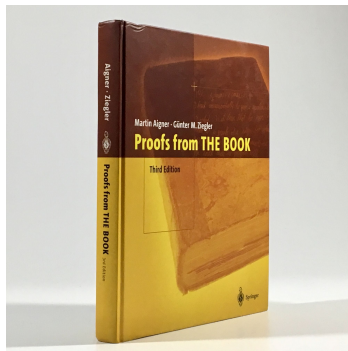
Theorem (Cayley, 1889)

The complete graph on vertex set $[n]$ has n^{n-2} spanning trees.

Paul Erdős and „Proofs from the book”

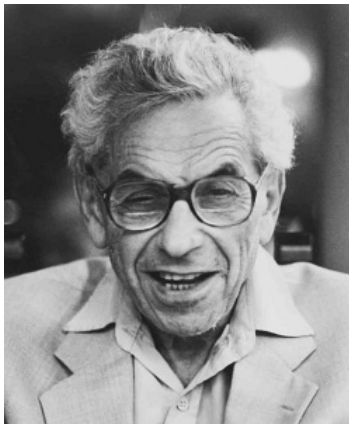


Paul Erdős (1913–1996)



M. Aigner, G.M. Ziegler,
Proofs from THE BOOK
6th edition, Springer 2018.

Paul Erdős and „Proofs from the book”



Paul Erdős (1913–1996)

Preface

Paul Erdős liked to talk about The Book, in which God maintains the perfect proofs for mathematical theorems, following the dictum of G. H. Hardy that there is no permanent place for ugly mathematics. Erdős also said that you need not believe in God but, as a mathematician, you should believe in The Book. A few years ago, we suggested to him to write up a first (and very modest) approximation to The Book. He was enthusiastic about the idea and, characteristically, went to work immediately, filling page after page with his suggestions. Our book was supposed to appear in March 1998 as a present to Erdős' 85th birthday. With Paul's unfortunate death in the summer of 1997, he is not listed as a co-author. Instead this book is dedicated to his memory.

We have no definition or characterization of what constitutes a proof from The Book: all we offer here is the examples that we have selected, hoping that our readers will share our enthusiasm about brilliant ideas, clever insights and wonderful observations. We also hope that our readers will enjoy this despite the imperfections of our exposition. The selection is to a great extent influenced by Paul Erdős himself. A large number of the topics were suggested by him, and many of the proofs trace directly back to him, or were initiated by his supreme insight in asking the right question or in making the right conjecture. So to a large extent this book reflects the views of Paul Erdős as to what should be considered a proof from The Book.



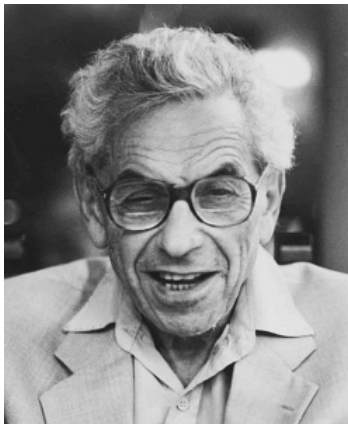
Paul Erdős



"The Book"

M. Aigner, G.M. Ziegler,
Proofs from THE BOOK
6th edition, Springer 2018.

Paul Erdős and „Proofs from the book”

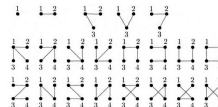


Paul Erdős (1913–1996)

Cayley's formula for the number of trees

Chapter 26

One of the most beautiful formulas in enumerative combinatorics concerns the number of labeled trees. Consider the set $N = \{1, 2, \dots, n\}$. How many different trees can we form on this vertex set? Let us denote this number by T_n . Enumeration "by hand" yields $T_1 = 1$, $T_2 = 1$, $T_3 = 3$, $T_4 = 16$, with the trees shown in the following table:



Arthur Cayley

Note that we consider labeled trees, that is, although there is only one tree of order 3 in the sense of graph isomorphism, there are 3 different labeled

M. Aigner, G.M. Ziegler,
Proofs from THE BOOK
6th edition, Springer 2018.



Counting trees — the Prüfer codes

The algorithm:

Input: a tree T on vertex set S , $S \subseteq \mathbb{N}$, $|S| = n$.

Output: code $f(T) = (v_1, v_2, \dots, v_{n-2})$, $v_i \in S$

Prüfer code(T)

```
 $T_1 := T;$  // a sequence of  $T$ -subtrees  
for  $i := 1$  until  $n - 2$  do  
   $x_i := \min\{x \in V(T_i) : \deg(x) = 1\};$  // the least leaf  
   $v_i :=$  the neighbour of  $x_i$  in  $T_i$ ;  
   $T_{i+1} := T_i \setminus x_i;$  // the subtree after removing  $x_i$   
 $code = (v_1, v_2, \dots, v_{n-2});$ 
```

Counting trees — the Prüfer codes



The algorithm:

Input: a tree T on vertex set S , $S \subseteq \mathbb{N}$, $|S| = n$.

Output: code $f(T) = (v_1, v_2, \dots, v_{n-2})$, $v_i \in S$



Theorem (Prüfer, 1918)

Function returning Prüfer codes is a bijection between sets of all trees on a given vertex set S into set S^{n-2} of $(n - 2)$ dimensional vectors from the set S .

Counting trees — the Prüfer codes

The decoding algorithm:

Input: The set $S \subseteq \mathbb{N}$, $|S| = n$ and a code $f = (v_1, v_2, \dots, v_{n-2})$,
 $v_i \in S$

Output: a tree $T(S, f)$ on vertex set S

Decode Prüfer(S , *code*)

$S_1 := S$; // a sequence of S -subsets

for $i := 1$ **until** $n - 2$ **do**

$x_i := \min S_i \setminus \{v_i, \dots, v_n\}$; // the least leaf

 // v_i is a neighbour of x_i in T ;

$S_{i+1} := S_i \setminus x_i$; // the subset after removing x_i

$T = (S, \{\{x_i, v_i\} : i = 1, 2, \dots, n - 2\} \cup S_{n-1})$;

// T has $n - 2$ edges of the form $\{x_i, v_i\}$ and one additional edge

// between two elements left in S_{n-1} .

Counting trees — the Renyi's approach



Idea: instead of counting trees let us count forests!

Let $T_{n,k}$ ($n, k \in \mathbb{N}$, $k \leq n$) denote the number of forests on vertex set $\{1, 2, \dots, n\}$, composed of k trees such that each vertex from the set $\{1, 2, \dots, k\}$ belongs to a different tree.



Theorem (Renyi, 1959)

For $n \geq k \geq 1$ one has

$$T_{n,k} = kn^{n-k-1},$$

in particular $T_{n,1} = n^{n-2}$.

Counting spanning trees of any graph

Theorem (Kirchhoff's matrix-tree theorem)

Let G be any connected, loopless multigraph with vertex set $V(G) = \{1, 2, \dots, n\}$. Let $Q = [q_{i,j}]_{i,j \leq n}$ be a matrix such that $(-q_{i,j})$ denotes the number of edges between vertices i and j (for $i \neq j$) while $q_{i,i} = \deg(i)$.

If Q^ is a matrix obtained from Q by removing the s -th row and the t -th column then the number of spanning trees of G is equal to $\tau(G) = (-1)^{s+t} \det Q^*$.*

Outline



- 1 Motivation
- 2 Cut-vertices and cut-edges
- 3 Definition and basic properties
- 4 Spanning trees and counting trees
- 5 Applications of trees**



Distance in a graph

Definition

Given graph G , for $u, v \in V(G)$ a **distance** $d(u, v)$ from u to v is the length of the shortest path from u to v , provided that some uv -path exists. Otherwise, let $d(u, v) = \infty$.

In a connected graph all distances are finite.

Definition

Let G be any connected graph. **Eccentricity** of a vertex $u \in V(G)$ is

$$\epsilon(u) = \max_{v \in V(G)} d(u, v).$$

Diameter & radius of graph G are defined by

$$\text{diam}(G) = \max_{u \in V(G)} \epsilon(u), \quad \text{rad}(G) = \min_{u \in V(G)} \epsilon(u).$$

Distance in a graph

Definition

Given graph G , for $u, v \in V(G)$ a **distance** $d(u, v)$ from u to v is the length of the shortest path from u to v , provided that some uv -path exists. Otherwise, let $d(u, v) = \infty$.

In a connected graph all distances are finite.

Definition

Let G be any connected graph. **Eccentricity** of a vertex $u \in V(G)$ is

$$\epsilon(u) = \max_{v \in V(G)} d(u, v).$$

Diameter & radius of graph G are defined by

$$\text{diam}(G) = \max_{u \in V(G)} \epsilon(u), \quad \text{rad}(G) = \min_{u \in V(G)} \epsilon(u).$$



Rooted trees

Definition

Rooted tree is a pair (T, r) , $r \in V(T)$ where T is a tree. Vertex r is called the **root** of tree T .

Because every two vertices in a tree are joined by exactly one path, in a rooted tree there exists a natural partial order of vertices: we say that vertex u precedes a vertex v ($u \preceq v$) when u lies on a path from the root to vertex v .

Definition

Given a rooted tree (T, r) and two vertices $u, v \in V(T)$ such that $u \preceq v$. We call u an **ancestor** of vertex v , and v a **descendant** of u . Moreover, when u and v are neighbours we call u a **parent** of v , and v a **child** of u .

A picture root up ;-).



Rooted trees

Definition

Rooted tree is a pair (T, r) , $r \in V(T)$ where T is a tree. Vertex r is called the **root** of tree T .

Because every two vertices in a tree are joined by exactly one path, in a rooted tree there exists a natural partial order of vertices: we say that vertex u precedes a vertex v ($u \preceq v$) when u lies on a path from the root to vertex v .

Definition

Given a rooted tree (T, r) and two vertices $u, v \in V(T)$ such that $u \preceq v$. We call u an **ancestor** of vertex v , and v a **descendant** of u . Moreover, when u and v are neighbours we call u a **parent** of v , and v a **child** of u .

A picture root up ;-).



Rooted trees

Definition

Rooted tree is a pair (T, r) , $r \in V(T)$ where T is a tree. Vertex r is called the **root** of tree T .

Because every two vertices in a tree are joined by exactly one path, in a rooted tree there exists a natural partial order of vertices: we say that vertex u precedes a vertex v ($u \preceq v$) when u lies on a path from the root to vertex v .

Definition

Given a rooted tree (T, r) and two vertices $u, v \in V(T)$ such that $u \preceq v$. We call u an **ancestor** of vertex v , and v a **descendant** of u . Moreover, when u and v are neighbours we call u a **parent** of v , and v a **child** of u .

A picture root up ;-).



Rooted trees

Definition

Let (T, r) be a rooted tree and $v \in V(T)$. Distance $d(r, v)$ is called a **level of vertex** v . Set of vertices with distance k from the root is called the k -th **level of the tree** T . Eccentricity of the root is called a **height of the tree** T .

A picture root up ;-).



Binary trees

Definition

Rooted tree (T, r) in which additionally childer of every vertex are numbered (properly) with numbers from the set $\{0, 1, \dots, m-1\}$ is called **m -ary tree**. A 2-ary tree is called a **binary tree**.

Left child, right child and a picture.

Selected applications of trees



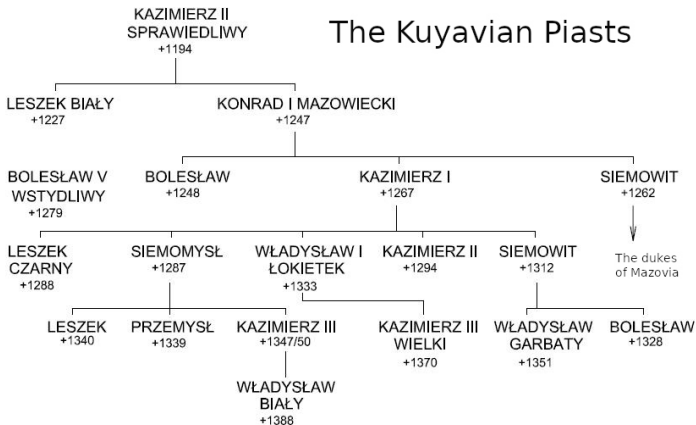
A binary search and binary search trees (BST).

Lexicographic order and dictionary trees (or prefix-suffix ones).

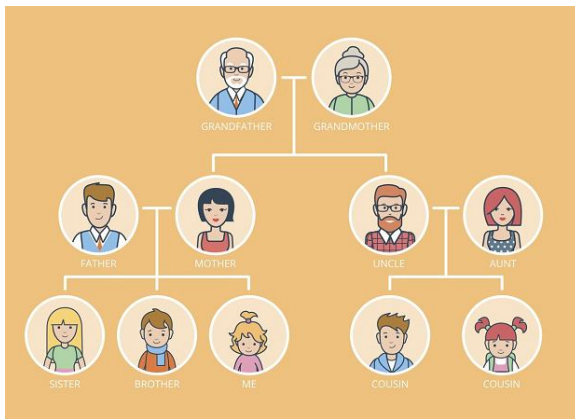
Family trees (genealogy)



The Kuyavian Piasts



Family trees (?) (genealogy)



Bonus: source blog.mrog.org (in Polish)



Poborowy Jerzy Kowalik
06-400 Ciechanów
ul. Sienkiewicza 7 m. 4

Do:
Naczelnik Sił Zbrojnych RP
Prezydent RP
Aleksander Kwaśniewski
Pałac Prezydencki
00-071 Warszawa
ul. Krakowskie Przedmieście 48/50

Ciechanów, 17.06.2002

Wielce Szanowny Panie Prezydencie,

Uprzejmie proszę o zwolnienie mnie z szaczytnego obowiązku pełnienia służby wojskowej ze względu na trudną sytuację rodzinną, w jakiej się znalazłem. A mianowicie:

Mam 24 lata i ożeniłem się w wdowę w wieku lat 44. Moja żona ma córkę w wieku lat 25. Tak się złożyło, że mój ojciec ożenił się z córką mojej żony. Tym samym, mój ojciec stał się moim zięciem, gdyż poślubił moją córkę. W związku z tym jest ona jednocześnie moją córką i macochą.

Mojej żonie i mnie urodził się w styczniu syn. To dziecko jest bratem żony mojego ojca, czyli jego szwagrem. Jednocześnie, będąc bratem mojej macochy jest moim wujkiem. Czyli mój syn jest moim wujkiem.

Żona mojego ojca, czyli moja córka, powiła na Wielkanoc chłopczyka, który jest jednocześnie moim bratem, gdyż jest synem mojego ojca, i wnukiem, ponieważ jest synem córki mojej żony.

Jestem więc bratem mojego wnuka, a będąc mężem teściowej ojca tego dziecka, pełnię jakby funkcję ojca mojego ojca, pozostając bratem jego syna. Jestem wobec tego własnym dziadkiem.

Dlatego też proszę uprzejmie Pana Prezydenta o zwolnienie mnie ze służby wojskowej, gdyż, o ile mi wiadomo, prawo nie zezwala powoływać do wojska jednocześnie dziadka, ojca i syna z jednej rodziny.

Wierząc w zrozumienie mojej sytuacji przez Pana Prezydenta pozostaję z pozowaniem

Jerzy Kowalik
JKowalik

Bonus: source blog.mrog.org (in Polish)

